

Guide d'animation

Atelier 1 · Je regarde, je choisis, j'enregistre — document réservé à la formatrice

Sommaire

Minutage	1
Matériel	2
Simulation papier (Temps 2)	2
Corrigés	3
Le vote du Temps 1	3
Le quiz éclair	3
L'enquête du Temps 3	3
Le tri de cartes du Temps 4	4
Chaque « À vous » en deux versions	4
Les messages de Git à ne pas craindre	5
Plan B	5
D'où viennent les sorties affichées	5

Le diaporama (page [slides.html](#) du site, touche **S** pour les notes) contient le discours slide par slide. Ce guide rassemble ce dont tu as besoin **autour** : le minutage d'ensemble, le matériel, les corrigés, les variantes démo/pratique, et quoi faire quand ça coince.

Minutage

Horloge	Durée	Séquence	Ce qui se passe
0:00	1 min	Ouverture	Accueil, services ouverts
0:01	8 min		Film « Le Nouveau »
0:09	2 min		« Qui s'est reconnu ? » — un mot chacun
0:11	1 min		Pires noms de fichiers · trois objectifs · cinq portes · le fil rouge
0:12	2 min	Temps 1	« Vendredi, mon script fonctionnait »
0:14	4 min		Vote a/b/c, débat en binôme, revote
0:18	5 min		L'image (l'album) · Sous le capot (instantané, empreinte)
0:23	2 min		La chaîne (trois commits de Marie)
0:25	2 min		À retenir, questions
0:27	2 min	Temps 2	<code>git init</code> , <code>git status</code> : qu'est-ce que je photographie ?
0:29	1 min		Les quatre phrases de Git traduites
0:30	4 min		L'image (la table) · les trois zones · <code>add</code> fige une version
0:34	5 min		Simulation papier : Git, c'est toi
0:39	6 min		À vous, étape 1 : la première photo
0:45	5 min		À vous, étape 2 : le seuil à 10 %
0:50	1 min		VS Code Source Control
0:51	3 min		Trois pièges provoqués, le message de commit
0:54	1 min		À retenir
0:55	3 min	Quiz	Quatre questions, 45 s chacune
0:58	5 min	Pause	Vérifier les dossiers de Marie
1:03	5 min	Temps 3	Marie est absente · L'image · Sous le capot (<code>log</code> , <code>show</code>)
1:08	8 min		Enquête en binômes
1:16	2 min		Réponses, <code>git show b0b1445</code> , « et si c'était modif ? »
1:18	3 min		Deux besoins, deux commandes · <code>restore</code> sous le capot

Horloge	Durée	Séquence	Ce qui se passe
1:21	5 min		À vous : <code>restore</code> sur README, puis <code>--source</code> (fusible)
1:26	2 min		Hors périmètre · À retenir
1:28	1 min	Temps 4	<code>git status</code> chez Marie : j'ajoute tout ?
1:29	4 min		Tri de cartes (fusible)
1:33	2 min		Critères, sous le capot, architecture
1:35	1 min		À vous : <code>.gitignore</code>
1:36	4 min	Temps 5	Service éphémère · L'image · Git ≠ GitLab · <code>origin, push, -u</code>
1:40	10 min		À vous : projet GitLab, <code>remote add, push</code> , lecture dans GitLab
1:50	2 min		Pièges · À retenir
1:52	2 min	Clôture	Les treize lignes, distribution du mémo
1:54	2 min		Ticket de sortie
1:56	4 min		Défi de la semaine · merci

Deux fusibles, si tu prends du retard :

1. **Temps 4** : saute le tri de cartes, donne directement le `.gitignore` (slide « Écrire un `.gitignore` ») et passe au Temps 5. Gain : 5 minutes.
2. **Temps 3** : le `git restore --source` passe en démonstration sur ton écran au lieu d'être tapé par tous. Gain : 2 à 3 minutes.

Ne sacrifie jamais le Temps 5 : c'est le seul moment où leur travail sort de leur service.

Matériel

- **Trois feuilles A4** titrées Bureau, Table de sélection, Album, et une table où les poser, visible de tous.
- **Des post-its** (un par fichier de l'étude : `README.md`, `rapport.qmd`, `01_import.R`, `02_nettoyage.R`, `03_analyse.R`, `brouillon_test.R`, `data/...csv`), un stylo, une agrafeuse.
- **Les 12 cartes de tri**, imprimées et découpées : un jeu par groupe de 2 ou 3 (`materiel/cartes-tri.pdf` sur le site).
- **Le mémo A4**, un par personne (`materiel/memo.pdf` sur le site).
- **Les tickets de sortie** : une demi-feuille par personne avec les trois lignes « Une chose que je sais faire / Une chose encore floue / La commande que je taperai demain ».
- **Le film** : accès YouTube vérifié, ou `video/le-nouveau.mp4` sur le poste (le dossier `video/` est ignoré par Git).
- **Le kit testé** sur un service neuf. Attention, une panne est **voulue** : dans `entreprises_defense_marie/`, `rapport.qmd` ne se rend pas et `03_analyse` s'arrête sur `Column `secteur` is not found` (commit 7, `b0b1445`). C'est la panne que le Temps 3 fait réparer : ne la corrige pas avant l'atelier. `entreprises_defense/`, lui, doit se rendre sans erreur. `kit/test_kit.sh` vérifie ce comportement sur `03_analyse` (quand R est installé).
- **Ton projet `entreprises_defense_marie` déjà poussé** sur le GitLab interne (plan B du Temps 5).

Simulation papier (Temps 2)

Tu joues Git : tu ne bouges rien de toi-même, tu n'agis que sur commande orale. Tous les post-its commencent sur Bureau.

1. « `git status` » → tu lis : « Je vois sept fichiers que je ne suis pas. »
2. « `git add README.md` » → tu **recopies** le post-it sur Table de sélection ; l'original reste sur Bureau.
3. Un participant gribouille le post-it `README.md` resté sur Bureau. « `git status` » → « `README.md` est sur la table, et il a changé sur le bureau depuis. »
4. « `git commit -m "..."` » → tu prends tout ce qui est sur la table, plus la dernière version de ce qui était déjà dans l'album, tu agrafes le tout comme une nouvelle page d'Album avec la légende. La table se vide.
5. « `git commit` » sans rien sur la table → « Rien à photographier. »

Corrigés

Le vote du Temps 1

(b) une copie complète du projet. Le commit est un instantané complet ; les fichiers inchangés ne sont pas recopiés parce que Git range chaque contenu sous son empreinte ; le diff (a) est calculé à la demande. Si quelqu'un objecte que Git compresse par différences : c'est vrai à l'empaquetage, mais c'est une optimisation de stockage invisible, le modèle reste la photo complète.

Le quiz éclair

1. **Que contient la photo ?** Tout le projet : la nouvelle version de `02_nettoyage.R`, et l'**ancienne** version de `rapport.qmd` (celle du commit précédent). La modification de `rapport.qmd` reste dans le répertoire de travail.
2. **La correction est-elle enregistrée ?** Non. `git add` a figé la version d'avant la correction ; `git status` montre le fichier dans les deux listes.
3. **L'étude est-elle à l'abri sur GitLab ?** Non. `git status` ne parle que du poste ; sans `git push`, GitLab n'a rien, et le service Onyxia peut disparaître.
4. **Deux commits peuvent-ils avoir la même empreinte ?** En pratique, non : l'empreinte porte sur tout le commit. Mathématiquement, des collisions SHA-1 existent (fabriquées exprès en 2017) ; Git détecte ces attaques et migre vers SHA-256. Par hasard, jamais.

L'enquête du Temps 3

Les empreintes de Marie, **produites par la version courante de `kit.sh` ; relancer `kit/test_kit.sh` après toute modification** (le test vérifie que les empreintes ci-dessous figurent toujours dans cette page) :

#	Date (2026)	Auteur	Message	Variante R	Variante Python
1	mar. 1 sept. 09:12	Marie	Initialise l'étude des entreprises liées à la défense	81bd623	c01700c
2	mer. 2 sept. 10:40	Marie	Ajoute l'import de l'extrait entreprises_defense_2024	3d6a872	d6a9b18
3	ven. 4 sept. 15:05	Marie	Ne garde que les entreprises dont la part de CA défense dépasse 20 %	af989d8	3948311
4	mar. 8 sept. 11:30	Marie	Ajoute la ventilation par taille d'entreprise (PME, ETI, GE)	842a418	b62bd95
5	jeu. 10 sept. 16:48	Franck	Corrige le calcul du CA défense (doublons de SIREN)	456a938	3c9fb14
6	ven. 11 sept. 17:20	Marie	Passe le graphique des effectifs en parts (%) à la demande du chef de bureau	155ae9a	693c70a
7	lun. 14 sept. 09:05	Marie	Ajoute la ventilation par secteur d'activité (NAF)	b0b1445	104a65d
8	mer. 16 sept. 14:10	Marie	Met à jour le rapport avec les résultats 2024	24e5e4a	c4a8e6b

Empreintes complètes

Variante R

```
81bd6237873727a79e027bf5c7e1d795b0a038b3 1 Initialise l'étude...
3d6a8722b4c203c6a1ba346894a8403567db5137 2 Ajoute l'import...
af989d85593715b68ab33fb55db0b453603e332c 3 Ne garde que...
842a418dadbc7a3df82204d7eaa546ffcd55b28 4 Ajoute la ventilation par taille...
456a938d55720bc818d8dedf6d43d0734287e903 5 Corrige le calcul du CA défense...
155ae9a6e9d9be8237c6c7b5e152b26f5a022187 6 Passe le graphique... en parts (%)
b0b1445b986bec6278113229a4c5b64e30c17b89 7 Ajoute la ventilation par secteur (NAF)
24e5e4acd405c1b7c61ef0953d5e14629a4c5c48 8 Met à jour le rapport...
```

Variante Python

```
c01700c2175660f9babbb766f109e183be86deb 1 Initialise l'étude...
```

d6a9b18c74d177019a5647e504f61163726b8d39	2 Ajoute l'import...
3948311611c9ca4f71b6847d1c5ae68c26b3e595	3 Ne garde que...
b62bd9527ef2073eb828adff5167ae73e439e683	4 Ajoute la ventilation par taille...
3c9fb1483f22a314cb114aec425d5134d91254e	5 Corrige le calcul du CA défense...
693c70a1e91dc3e6760af2ea89ba044061f2a63c	6 Passe le graphique... en parts (%)
104a65db0c1d6e2a8e6e5963e1af65515fbc712e	7 Ajoute la ventilation par secteur (NAF)
c4a8e6b67bc015a8a0f4108926b29c1cf60ffe4b	8 Met à jour le rapport...

Réponses attendues : le passage en parts, c'est le **commit 6** — Marie, vendredi 11 septembre à 17 h 20, « à la demande du chef de bureau ». Le script cassé, c'est le **commit 7** — lundi 14 septembre, `group_by(secteur)` alors que la colonne `secteur` n'existe pas (seule `naf` existe).

Si R est disponible sur ton service, tu peux montrer l'erreur en direct : `Rscript scripts/03_analyse.R` dans `entreprises_defense_marie/` échoue sur la colonne `secteur` ; après le `git restore --source`, il fonctionne. `kit/test_kit.sh` vérifie ce comportement (échec au commit 8, succès au commit 6) quand R est installé.

Les notes de réunion laissées chez Marie (`notes_reunion_chef_de_bureau.md`) confirment l'histoire, si un binôme les ouvre : le chef de bureau a demandé les parts, et la ventilation NAF était « à voir ».

Le tri de cartes du Temps 4

Carte	Pile	Pourquoi
<code>scripts/02_nettoyage.R</code>	à versionner	Du code, du texte, le cœur de l'étude
<code>rapport.qmd</code>	à versionner	La source du rapport, du texte
<code>README.md</code>	à versionner	La documentation
<code>.gitignore</code>	à versionner	Il se partage avec l'équipe
<code>data/entreprises_defense_2024.csv</code> (extrait synthétique)	discutable	Petit, texte, non confidentiel : versionnable. Mais la règle générale reste : les données hors de Git
<code>data/bitd_2024.parquet</code> (600 Mo)	à ignorer	Binaire, volumineux, sa place est sur S3
<code>rapport.html</code>	à ignorer	Régénérable par <code>quarto render</code>
<code>figures/effectifs_par_taille.png</code>	discutable	Régénérable par <code>03_analyse.R</code> : à ignorer, sauf s'il s'agit d'une pièce livrée à une date donnée
<code>.Rhistory</code>	à ignorer	Propre à chaque poste, sans valeur pour l'équipe
<code>.Renviron</code> (contient un jeton)	jamais	Secret : un jeton commité reste dans l'historique ; le révoquer aussitôt
<code>donnees_entreprises_confidentiel.csv</code>	jamais	Données individuelles d'entreprises : secret statistique
<code>notes_reunion_chef_de_bureau.md</code>	discutable	Du texte utile, mais nominatif et interne : pas dans le dépôt de l'étude

Chaque « À vous » en deux versions

Moment	Version pratique (par défaut)	Version démo (si le groupe décroche ou si le temps manque)
Temps 2, étape 1	Binômes pilote/copilote, prédiction avant chaque <code>git status</code>	Tu tapes, la salle prédit à voix haute
Temps 2, étape 2	Les binômes échangent les rôles, même protocole	Tu modifies le seuil en partageant ton écran
Temps 2, pièges	Chacun provoque les trois pièges	Tu les provoques, ils lisent la sortie avec toi
Temps 3, enquête	Binômes, 8 min, aide graduée	Tu cherches en direct, ils te guident
Temps 3, <code>restore</code>	Tout le monde : <code>restore README.md</code> puis <code>--source</code>	<code>restore README.md</code> pour tous, <code>--source</code> en démo (fusible)
Temps 4, tri	Groupes de 2 ou 3, cartes papier	Tri collectif à main levée, carte par carte (ou fusible : <code>.gitignore</code> fourni)

Moment	Version pratique (par défaut)	Version démo (si le groupe décroche ou si le temps manque)
Temps 4, <code>.gitignore</code>	Chacun crée le fichier dans VS Code	Tu le crées, ils le copient depuis le guide
Temps 5, push	Chacun crée son projet et pousse	Tu pousse ton projet ; ceux qui bloquent suivent sur ton écran

Démo facultative (2 min maximum, Temps 3, jamais sur les slides) : si quelqu'un veut voir tout le projet tel qu'il était vendredi, `git switch --detach 155ae9a`, on regarde les fichiers dans VS Code, puis `git switch main` pour revenir. Ne le fais pas taper.

Les messages de Git à ne pas craindre

Ce qu'on voit	Ce que ça veut dire	Quoi faire
Un éditeur plein écran, des ~ sur la gauche	<code>git commit</code> sans <code>-m</code> : Git demande le message dans l'éditeur	Échap, puis <code>:q!</code> , puis Entrée ; recommencer avec <code>-m "..."</code>
: ou (END) en bas de l'écran après <code>git log</code> ou <code>git show</code>	Le lecteur de texte de Git (la sortie dé-passe l'écran)	Flèches pour défiler, q pour quitter
no changes added to commit	Commit sans add	<code>git add <fichier></code> , puis <code>git commit</code>
nothing added to commit but untracked files present	Rien n'est sélectionné ; il reste des fichiers non suivis	Normal s'ils sont volontairement laissés dehors
fatal: not a git repository	On n'est pas dans le bon dossier	<code>cd ~/work/atelier-git/entreprises_defense</code> (ou <code>..._marie</code>)
hint: Using 'master' as the name for the initial branch	Le kit n'a pas été lancé avant <code>git init</code>	Pousser avec <code>git push -u origin master</code> , ou relancer le kit sur un service neuf
! [rejected] ... (fetch first)	Projet GitLab créé avec un README	Supprimer et recréer le projet sans README
Une demande de Username / Password au push	Jetons absent ou expiré	Vérifier Mon compte sur Onyxia, relancer un service ; sinon plan B

Plan B

- **Onyxia indisponible** (panne, services qui ne démarrent pas) : tout l'atelier bascule sur le bac à sable du site ([bac-a-sable.html](#)), dans le navigateur. Chacun y ouvre la même étude que le kit, en R ou en Python. La prise en main couvre le Temps 2 ; les missions 1 à 3 le diff, le commit cohérent et le piège du `add` ; la mission 4 le `git restore` du Temps 3. Le `.gitignore` du Temps 4 y est visible (données et sorties ignorées), le tri de cartes se fait sur papier, le Temps 5 en démonstration sur ton écran.
- **Le push échoue** pour une partie de la salle : montre ton projet `entreprises_defense_marie` déjà poussé sur le GitLab interne (historique, un commit, son diff, le README) ; les personnes concernées suivent sur ton écran et referont le push chez elles (défi de la semaine).
- **Pas de YouTube dans la salle** : [video/le-nouveau.mp4](#) sur ton poste.
- **Service recréé le jour J** : `bash kit.sh --force` (le kit est idempotent : mêmes dossiers, mêmes empreintes).

D'où viennent les sorties affichées

Toutes les sorties des slides et du guide ont été produites en exécutant réellement les commandes sur le kit (Git 2.43, identité fictive « Camille », dates fixées pour la reproductibilité). Deux exceptions, signalées :

- la sortie de `git push` a été obtenue avec un dépôt distant local ; seule l'adresse a été remplacée par celle de GitLab du GitLab interne ;
- la slide « Et si le message avait été modif ? » montre un historique fictif, fabriqué pour l'exemple, ce que tu dis à voix haute.

Les empreintes des commits des participants (Temps 2, 4 et 5) dépendent de leur nom et de l'heure : elles diffèrent d'un poste à l'autre. Celles de Marie sont identiques partout.